

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

дисциплина: Архитектура компьютера

Студент: Ксения Танчила

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

Список иллюстраций.....	3
Цель работы.....	4
Задание.....	5
Теоритическое введение.....	6
Выполнение лабораторной работы.....	7
Символьные и численные данные в NASM.....	7
Выполнение арифметических операций в NASM.....	11
Задания для самостоятельной работы.....	15
Выводы.....	17
Список литературы.....	18

Список иллюстраций

Рис. 4.1.1 — Создание каталога.....	7
Рис. 4.1.2 — Текст программы lab6-1.asm.....	7
Рис. 4.1.3 — Вывод программы lab6-1.asm.....	8
Рис. 4.1.4 — Исправленный текст программы lab6-1.asm.....	8
Рис. 4.1.5 — Вывод программы lab6-1.asm.....	9
Рис. 4.1.6 — Текст программы lab6-2.asm.....	9
Рис. 4.1.7 — Вывод программы lab6-2.asm.....	9
Рис. 4.1.8 — Исправленный текст программы lab6-1.asm.....	10
Рис. 4.1.9 — Вывод программы lab6-2.asm.....	10
Рис. 4.1.10 — Исправленный текст программы lab6-2.asm.....	10
Рис. 4.1.11 — Вывод программы lab6-2.asm.....	11
Рис. 4.2.1 — Создание файла lab6-3.asm.....	11
Рис. 4.2.2 — Текст программы lab6-3.asm.....	11
Рис. 4.2.3 — Вывод программы lab6-3.asm.....	12
Рис. 4.2.4 — Исправленный текст программы lab6-3.asm.....	12
Рис. 4.2.5 — Вывод программы lab6-3.asm.....	12
Рис. 4.2.6 — Создание файла variant.asm.....	13
Рис. 4.2.7 — Текст программы variant.asm.....	13
Рис. 4.2.8 — Вывод программы variant.asm.....	13
Рис. 4.3 — Результат выполнения самостоятельного задания.....	16

1 Цель работы

Освоение арифметических инструкций языка ассемблера NASM.

2 Задание

1. Символьные и численные данные в NASM
2. Выполнение арифметических операций в NASM
3. Выполнение заданий для самостоятельной работы

3 Теоретическое введение

Большинство инструкций на языке ассемблера требуют обработки операндов. Адрес операнда предоставляет место, где хранятся данные, подлежащие обработке. Это могут быть данные хранящиеся в регистре или в ячейке памяти. - Регистровая адресация – операнды хранятся в регистрах и в команде используются имена этих регистров, например: `mov ax,bx`. - Непосредственная адресация – значение операнда задается непосредственно в команде, Например: `mov ax,2`. - Адресация памяти – операнд задает адрес в памяти. В команде указывается символическое обозначение ячейки памяти, над содержимым которой требуется выполнить операцию. Ввод информации с клавиатуры и вывод её на экран осуществляется в символьном виде. Кодирование этой информации производится согласно кодовой таблице символов ASCII. ASCII – сокращение от American Standard Code for Information Interchange (Американский стандартный код для обмена информацией). Согласно стандарту ASCII каждый символ кодируется одним байтом. Среди инструкций NASM нет такой, которая выводит числа (не в символьном виде). Поэтому, например, чтобы вывести число, надо предварительно преобразовать его цифры в ASCII-коды этих цифр и выводить на экран эти коды, а не само число. Если же выводить число на экран непосредственно, то экран воспримет его не как число, а как последовательность ASCII-символов – каждый байт числа будет воспринят как один ASCII-символ – и выведет на экран эти символы. Аналогичная ситуация происходит и при вводе данных с клавиатуры. Введенные данные будут представлять собой символы, что сделает невозможным получение корректного результата при выполнении над ними арифметических операций. Для решения этой проблемы необходимо проводить преобразование ASCII символов в числа и обратно.

4 Выполнение лабораторной работы

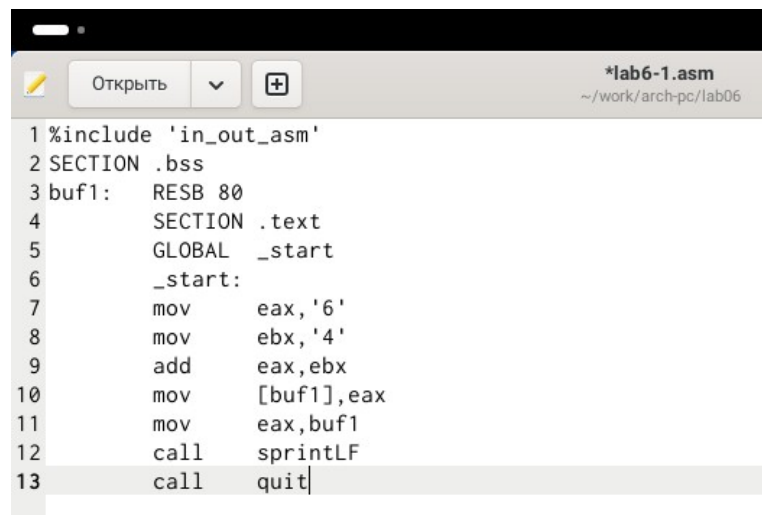
4.1 Символьные и численные данные в NASM

1. Создала каталог для программ лабораторной работы № 6, перешла в него и создала файл lab6-1.asm (Рис. 4.1.1).

```
.stanchila@dk6n17 ~/work/arch-pc/lab05 $ mkdir ~/work/arch-pc/lab06
.stanchila@dk6n17 ~/work/arch-pc/lab05 $ cd ~/work/arch-pc/lab06
.stanchila@dk6n17 ~/work/arch-pc/lab06 $ touch lab6-1.asm
```

Рис. 4.1.1 — Создание каталога

2. Рассмотрели примеры программ вывода символьных и численных значений. Программы выводили значения записанные в регистр eax. Ввела в файл lab6-1.asm текст программы (Рис, 4.1.2). В данной программе в регистр eax записывается символ 6 (mov eax,'6'), в регистр ebx символ 4 (mov ebx,'4'). Далее к значению в регистре eax прибавляем значение регистра ebx (add eax,ebx, результат сложения запишется в регистр eax). Далее выводим результат. Так как для работы функции sprintf в регистр eax должен быть записан адрес, использовали дополнительную переменную. Для этого записали значение регистра eax в переменную buf1 (mov [buf1],eax), а затем записали адрес переменной buf1 в регистр eax (mov eax,buf1) и вызвали функцию sprintf. Создала исполняемый файл и запустила его (Рис. 4.1.3).



```
*lab6-1.asm
~/work/arch-pc/lab06

1 %include 'in_out_asm'
2 SECTION .bss
3 buf1:  RESB 80
4         SECTION .text
5         GLOBAL _start
6         _start:
7         mov     eax,'6'
8         mov     ebx,'4'
9         add     eax,ebx
10        mov     [buf1],eax
11        mov     eax,buf1
12        call    sprintf
13        call    quit
```

Рис. 4.1.2 — Текст программы lab6-1.asm

```

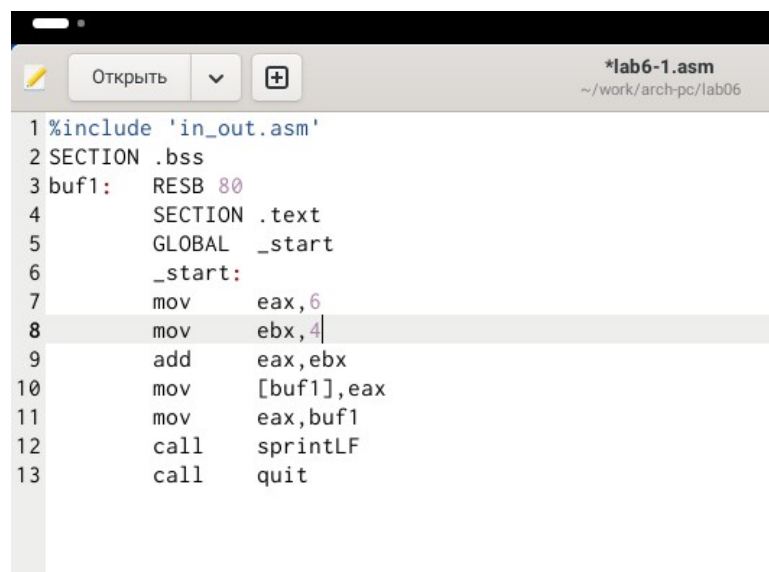
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-1 lab6-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-1
j

```

Рис. 4.1.3 — Вывод программы lab6-1.asm

В данном случае при выводе значения регистра `eax` мы ожидали увидеть число 10. Однако результатом был символ `j`. Это происходит потому, что код символа 6 равен 00110110 в двоичном представлении (или 54 в десятичном представлении), а код символа 4 – 00110100 (52). Команда `add eax,ebx` записала в регистр `eax` сумму кодов – 01101010 (106), что в свою очередь является кодом символа `j`.

3. Далее изменим текст программы и вместо символов, запишем в регистры числа. Исправьте текст программы `lab6-1.asm` следующим образом: заменила строки `mov eax,'6'` `mov ebx,'4'` на строки `mov eax,6` `mov ebx,4` (Рис. 4.1.4).



```

1 %include 'in_out.asm'
2 SECTION .bss
3 buf1:  RESB 80
4     SECTION .text
5     GLOBAL _start
6     _start:
7     mov     eax,6
8     mov     ebx,4
9     add     eax,ebx
10    mov     [buf1],eax
11    mov     eax,buf1
12    call    sprintfLF
13    call    quit

```

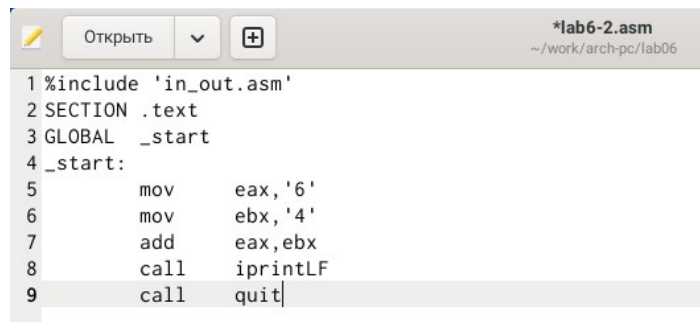
Рис. 4.1.4 — Исправленный текст программы lab6-1.asm

Создала исполняемый файл и запустила его. Как и в предыдущем случае при исполнении программы мы не получили число 10. В данном случае выводится символ с кодом 10. Символ с кодом 10 в таблице ASCII — это LF (Line Feed), что означает "перевод строки" или "новая строка". Этот символ не отображается на экране (Рис. 4.1.5).


```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-1 lab6-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-1
```

Рис. 4.1.5 — Вывод программы lab6-1.asm

4. Как отмечалось выше, для работы с числами в файле `in_out.asm` реализованы подпрограммы для преобразования ASCII символов в числа и обратно. Преобразовали текст программы `lab6-1.asm` с использованием этих функций. Создала файл `lab6-2.asm` в каталоге `~/work/arch-pc/lab06` и ввела в него текст программы `lab6-2.asm` (Рис. 4.1.6). Создала исполняемый файл и запустила его (Рис. 4.1.7).



```
*lab6-2.asm
~/work/arch-pc/lab06

1 %include 'in_out.asm'
2 SECTION .text
3 GLOBAL _start
4 _start:
5     mov     eax, '6'
6     mov     ebx, '4'
7     add     eax, ebx
8     call    iprintLF
9     call    quit
```

Рис. 4.1.6 — Текст программы lab6-2.asm

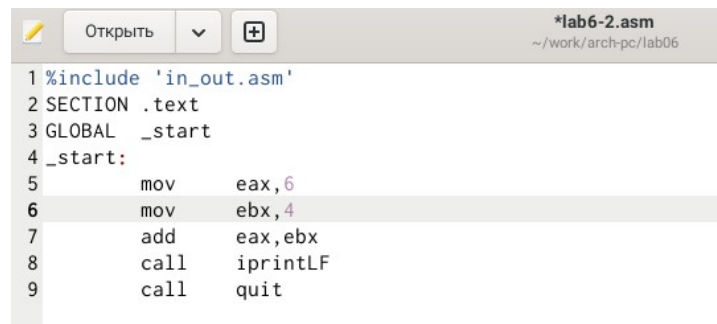
```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ touch lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-2 lab6-2.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-2
106
```

Рис. 4.1.7 — Вывод программы lab6-2.asm

В результате работы программы мы получили число 106. В данном случае, как и в первом, команда `add` складывает коды символов '6' и '4' ($54+52=106$). Однако, в отличие от программы из листинга 6.1, функция `iprintLF` позволяет вывести число, а не символ, кодом которого является это число.

5. Аналогично предыдущему примеру изменила символы на числа. Заменяла

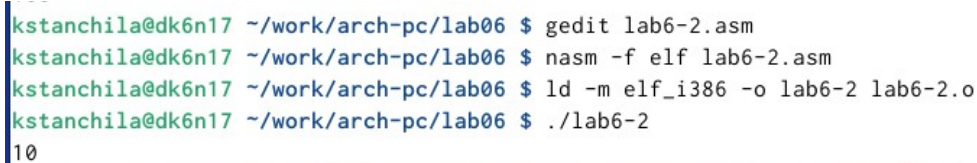
строки `mov eax,'6'` `mov ebx,'4'` на строки `mov eax,6` `mov ebx,4` (Рис. 4.1.8). Создала исполняемый файл и запустила его (Рис. 4.1.9).



```
*lab6-2.asm
~/work/arch-pc/lab06

1 %include 'in_out.asm'
2 SECTION .text
3 GLOBAL _start
4 _start:
5     mov     eax,6
6     mov     ebx,4
7     add     eax,ebx
8     call    iprintLF
9     call    quit
```

Рис. 4.1.8 — Исправленный текст программы lab6-2.asm

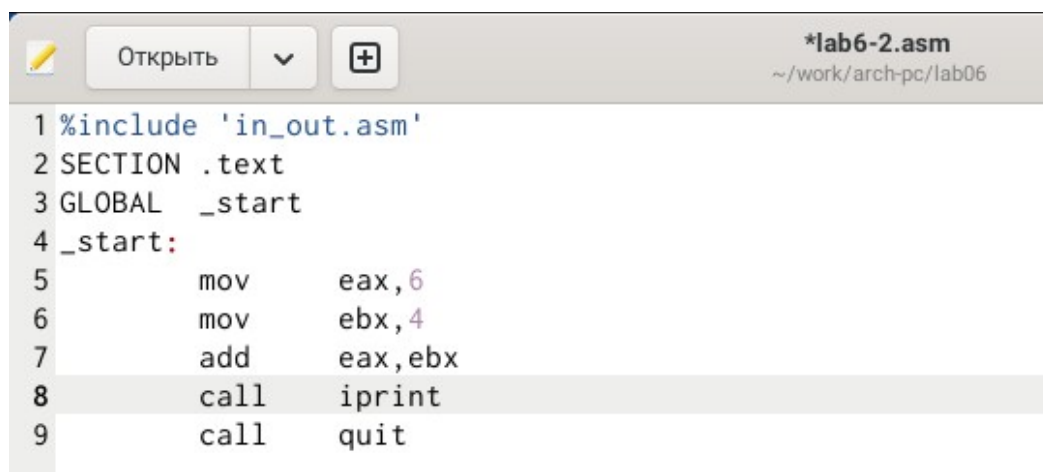


```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-2 lab6-2.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-2
10
```

Рис. 4.1.9 — Вывод программы lab6-2.asm

При выполнении программы получили результат 10.

Заменяла функцию `iprintLF` на `iprint` (Рис. 4.1.10). Создала исполняемый файл и запустила его (Рис. 4.1.11). Чем отличается вывод функций `iprintLF` и `iprint`?



```
*lab6-2.asm
~/work/arch-pc/lab06

1 %include 'in_out.asm'
2 SECTION .text
3 GLOBAL _start
4 _start:
5     mov     eax,6
6     mov     ebx,4
7     add     eax,ebx
8     call    iprint
9     call    quit
```

Рис. 4.1.10 — Исправленный текст программы lab6-2.asm

```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-2 lab6-2.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-2
10kstanchila@dk6n17 ~/work/arch-pc/lab06 $ touch ~/work/arch-pc/lab06/lab6-3.asm
```

Рис. 4.1.11 — Вывод программы lab6-2.asm

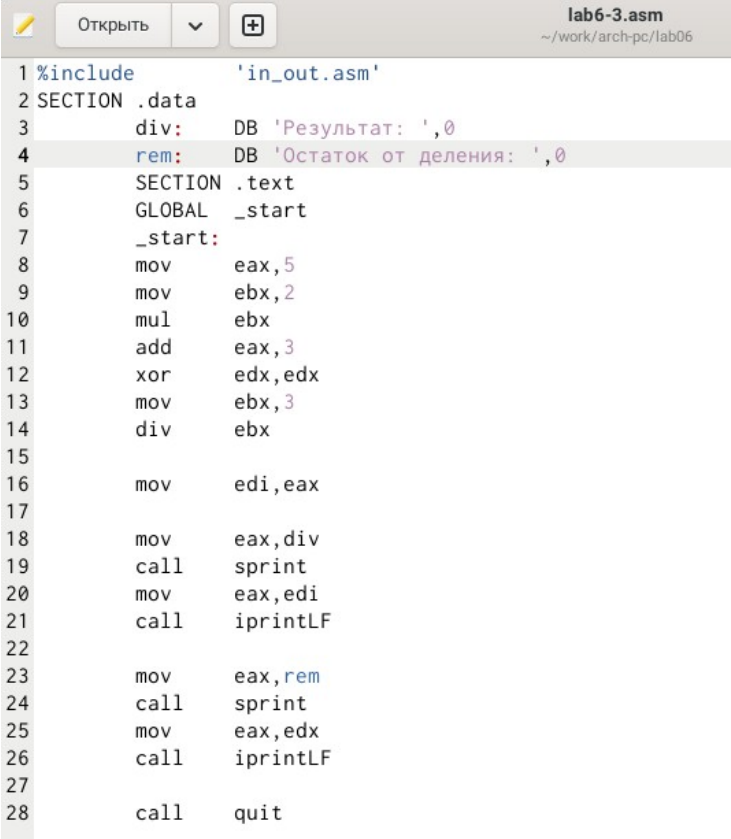
4.2 Выполнение арифметических операций в NASM

6. В качестве примера выполнения арифметических операций в NASM привела программу вычисления арифметического выражения $f(x) = (5 * 2 + 3)/3$. Создала файл lab6-3.asm в каталоге lab06 (Рис. 4.2.1).

```
10kstanchila@dk6n17 ~/work/arch-pc/lab06 $ touch ~/work/arch-pc/lab06/lab6-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-3.asm
```

Рис. 4.2.1 — Создание файла lab6-3.asm

Внимательно изучила текст программы из листинга 6.3 и ввела в lab6-3.asm (Рис. 4.2.2).



```
lab6-3.asm
~/work/arch-pc/lab06

1 %include 'in_out.asm'
2 SECTION .data
3     div: DB 'Результат: ',0
4     rem: DB 'Остаток от деления: ',0
5
6 SECTION .text
7 GLOBAL _start
8 _start:
9     mov     eax,5
10    mov     ebx,2
11    mul     ebx
12    add     eax,3
13    xor     edx,edx
14    mov     ebx,3
15    div     ebx
16    mov     edi,eax
17
18    mov     eax,div
19    call    sprint
20    mov     eax,edi
21    call    iprintLF
22
23    mov     eax,rem
24    call    sprint
25    mov     eax,edx
26    call    iprintLF
27
28    call    quit
```

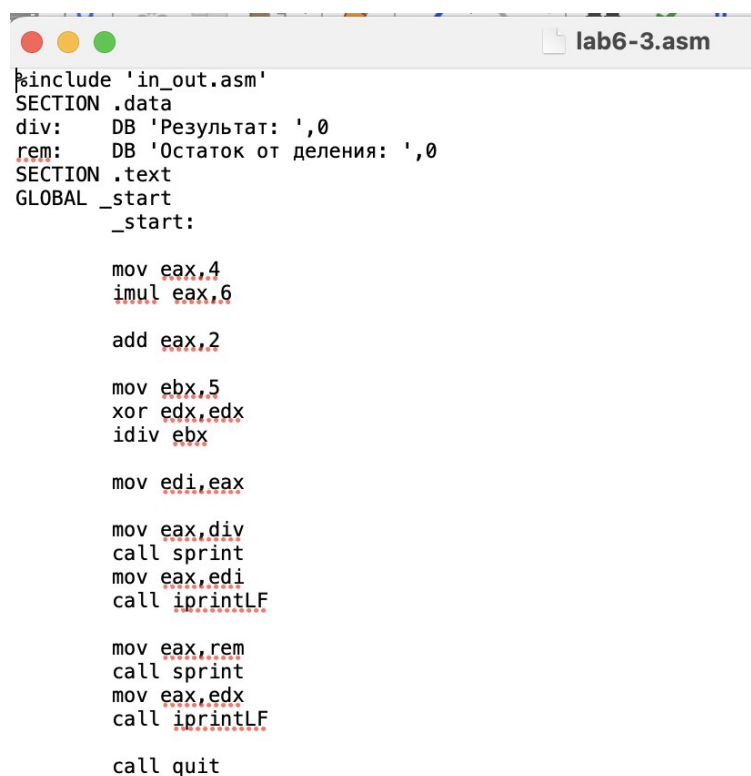
Рис. 4.2.2 — Текст программы lab6-3.asm

Создала исполняемый файл и запустила его (Рис. 4.2.3).

```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-3 lab6-3.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-3
Результат: 4
Остаток от деления: 1
kstanchila@dk6n17 ~/work/arch-pc/lab06 $
```

Рис. 4.2.3 — Вывод программы lab6-3.asm

Изменила текст программы для вычисления выражения $f(x) = (4 * 6 + 2)/5$ (Рис. 4.2.4). Создала исполняемый файл и проверила его работу (Рис. 4.2.5).



```
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
rem: DB 'Остаток от деления: ',0
SECTION .text
GLOBAL _start
_start:

    mov eax,4
    imul eax,6

    add eax,2

    mov ebx,5
    xor edx,edx
    idiv ebx

    mov edi,eax

    mov eax,div
    call sprint
    mov eax,edi
    call iprintLF

    mov eax,rem
    call sprint
    mov eax,edx
    call iprintLF

    call quit
```

Рис. 4.2.4 — Исправленный текст программы lab6-3.asm

```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab6-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf lab6-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o lab6-3 lab6-3.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./lab6-3
Результат: 5
Остаток от деления: 1
```

Рис. 4.2.5 — Вывод программы lab6-3.asm

7. В качестве другого примера рассмотрели программу вычисления варианта задания по номеру студенческого билета, работающую по следующему алгоритму:

- вывести запрос на введение № студенческого билета
- вычислить номер варианта по формуле: $(Sn \bmod 20) + 1$, где Sn – номер студенческого билета (В данном случае $a \bmod b$ – это остаток от деления a на b).
- вывести на экран номер варианта.

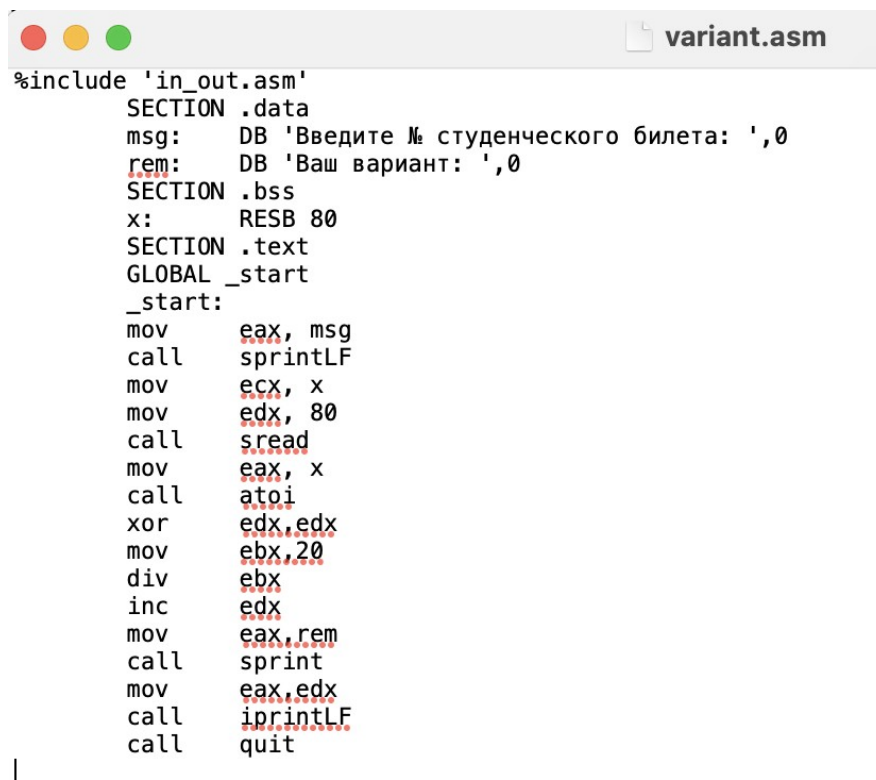
В данном случае число, над которым необходимо проводить арифметические операции, вводится с клавиатуры. Как отмечалось выше ввод с клавиатуры осуществляется в символьном виде и для корректной работы арифметических операций в NASM символы необходимо преобразовать в числа. Для этого может быть использована функция `atoi` из файла `in_out.asm`.

Создала файл `variant.asm` в каталоге `lab06` (Рис. 4.2.6).

```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ touch ~/work/arch-pc/lab06/variant.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit variant.asm
```

Рис. 4.2.6 — Создание файла `variant.asm`

Внимательно изучила текст программы и ввела в файл `variant.asm` (Рис. 4.2.7).



```
%include 'in_out.asm'
SECTION .data
msg:    DB 'Введите № студенческого билета: ',0
rem:    DB 'Ваш вариант: ',0
SECTION .bss
x:      RESB 80
SECTION .text
GLOBAL _start
_start:
mov     eax, msg
call    sprintLF
mov     ecx, x
mov     edx, 80
call    sread
mov     eax, x
call    atoi
xor     edx, edx
mov     ebx, 20
div     ebx
inc     edx
mov     eax, rem
call    sprint
mov     eax, edx
call    iprintLF
call    quit
```

Рис. 4.2.7 — Текст программы `variant.asm`

Создала исполняемый файл и запустила его. Проверила результат работы программы вычислив номер варианта аналитически (Рис. 4.2.8).

```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit variant.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf variant.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o variant variant.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./variant.asm
bash: ./variant.asm: Отказано в доступе
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./variant
Введите № студенческого билета:
1032253536
Ваш вариант: 17
kstanchila@dk6n17 ~/work/arch-pc/lab06 $
```

Рис. 4.2.8 — Вывод программы variant.asm

Ответы на вопросы:

1. Какие строки листинга 6.4 отвечают за вывод на экран сообщения ‘Ваш вариант:’?

Ответ:

mov eax,rem

call sprint

2. Для чего используются следующие инструкции? mov ecx, x mov edx, 80 call sread

Ответ:

эти строки кода отвечают за чтение ввода пользователя с клавиатуры.

3. Для чего используется инструкция “call atoi”?

Ответ:

эта инструкция используется для преобразования ASCII кода в число.

4. Какие строки листинга 6.4 отвечают за вычисления варианта?

Ответ:

mov eax,x

call atoi

xor edx,edx

mov ebx,20

div ebx

inc edx

5. В какой регистр записывается остаток от деления при выполнении инструкции “div ebx”?

Ответ:

в edx.

6. Для чего используется инструкция “inc edx”?

Ответ:

увеличивает edx на 1.

7. Какие строки листинга 6.4 отвечают за вывод на экран результата вычислений?

Ответ:

mov eax,rem

call sprint

mov eax,edx

call iprintLF

4.3 Задание для самостоятельной работы

1. Написала программу вычисления выражения $y = f(x)$. Программа должна выводить выражение для вычисления, выводить запрос на ввод значения x , вычислять заданное выражение в зависимости от введенного x , выводить результат вычислений. Вид функции $f(x)$ выбрала из таблицы 6.3 вариантов заданий в соответствии с номером полученным при выполнении лабораторной работы (17): $18(x + 1)/6$. Создала исполняемый файл и проверила его работу для значений x_1 и x_2 из 6.3: 3,1 (Рис. 4.3).


```
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit homework.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab5-21.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit lab5-2.asm.save.3
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ gedit homework.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ nasm -f elf homework.asm
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ld -m elf_i386 -o homework homework.o
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./homework
Выражение для вычисления:  $18 \cdot (x+1) / 6$     Введите x:
5
Результат вычисления равен: 18
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./homework
Выражение для вычисления:  $18 \cdot (x+1) / 6$     Введите x:
3
Результат вычисления равен: 12
kstanchila@dk6n17 ~/work/arch-pc/lab06 $ ./homework
Выражение для вычисления:  $18 \cdot (x+1) / 6$     Введите x:
1
Результат вычисления равен: 6
kstanchila@dk6n17 ~/work/arch-pc/lab06 $
```

Рис. 4.3 — Результат выполнения самостоятельного задания

6 Выводы

При выполнении данной лабораторной работы я освоила арифметические инструкции языка ассемблера NASM.

Список литературы

1. Пример выполнения лабораторной работы
2. Курс на ТУИС
3. Лабораторная работа №6
4. Программирование на языке ассемблера NASM Столяров А. В